

Node Similarity Algorithm

Neo4j Graph Data Science Library

Introduction

The Node Similarity algorithm compares a set of nodes based on the nodes they are connected to. Two nodes are considered similar if they **share many of the same neighbors**. Node Similarity computes pair-wise similarities based on the Jaccard metric, also known as the Jaccard Similarity Score, the Overlap coefficient, also known as the Szymkiewicz–Simpson coefficient, and the Cosine Similarity score. The first two are most frequently associated with unweighted sets, whereas Cosine with weighted input.

How It Works

The Node Similarity algorithm compares each node that has outgoing relationships with each other such node.

- For every node n , we collect the outgoing neighborhood $N(n)$ of that node, that is, all nodes m such that there is a relationship from n to m .
- For each pair n, m , the algorithm computes a similarity for that pair that equals the outcome of the selected similarity metric for $N(n)$ and $N(m)$.

In order to bound memory usage you can specify an explicit limit on the number of results to output per node, this is the 'topK' parameter.

Configuration Parameters

Parameter	Default	Description
similarityCutoff	1e-42	Lower bound for scores
degreeCutoff	1	Min node degree to include
upperDegreeCutoff	2147483647	Max node degree to include
topK	10	Top K scores per node
bottomK	10	Bottom K scores per node
topN	0	Global top N limit
bottomN	0	Global bottom N limit
similarityMetric	JACCARD	JACCARD, OVERLAP, COSINE

Execution Modes

Mode	Description
<code>stream</code>	Returns similarity scores directly
<code>stats</code>	Returns summary statistics
<code>mutate</code>	Writes results to in-memory graph
<code>write</code>	Persists results to Neo4j database
<code>estimate</code>	Estimates memory requirements

Stream Mode Syntax

```
CALL gds.nodeSimilarity.stream(  
  graphName: String,  
  configuration: Map  
)  
YIELD node1: Integer, node2: Integer, similarity: Float
```

Returns:

- `node1` , `node2` — Node IDs of the pair
- `similarity` — Computed similarity score

Stats Mode

```
CALL gds.nodeSimilarity.stats('myGraph')  
YIELD nodesCompared, similarityPairs,  
       similarityDistribution
```

Returns aggregate metrics:

- `nodesCompared` — count of nodes evaluated
- `similarityPairs` — number of pairs found
- `similarityDistribution` — min, max, mean, percentiles

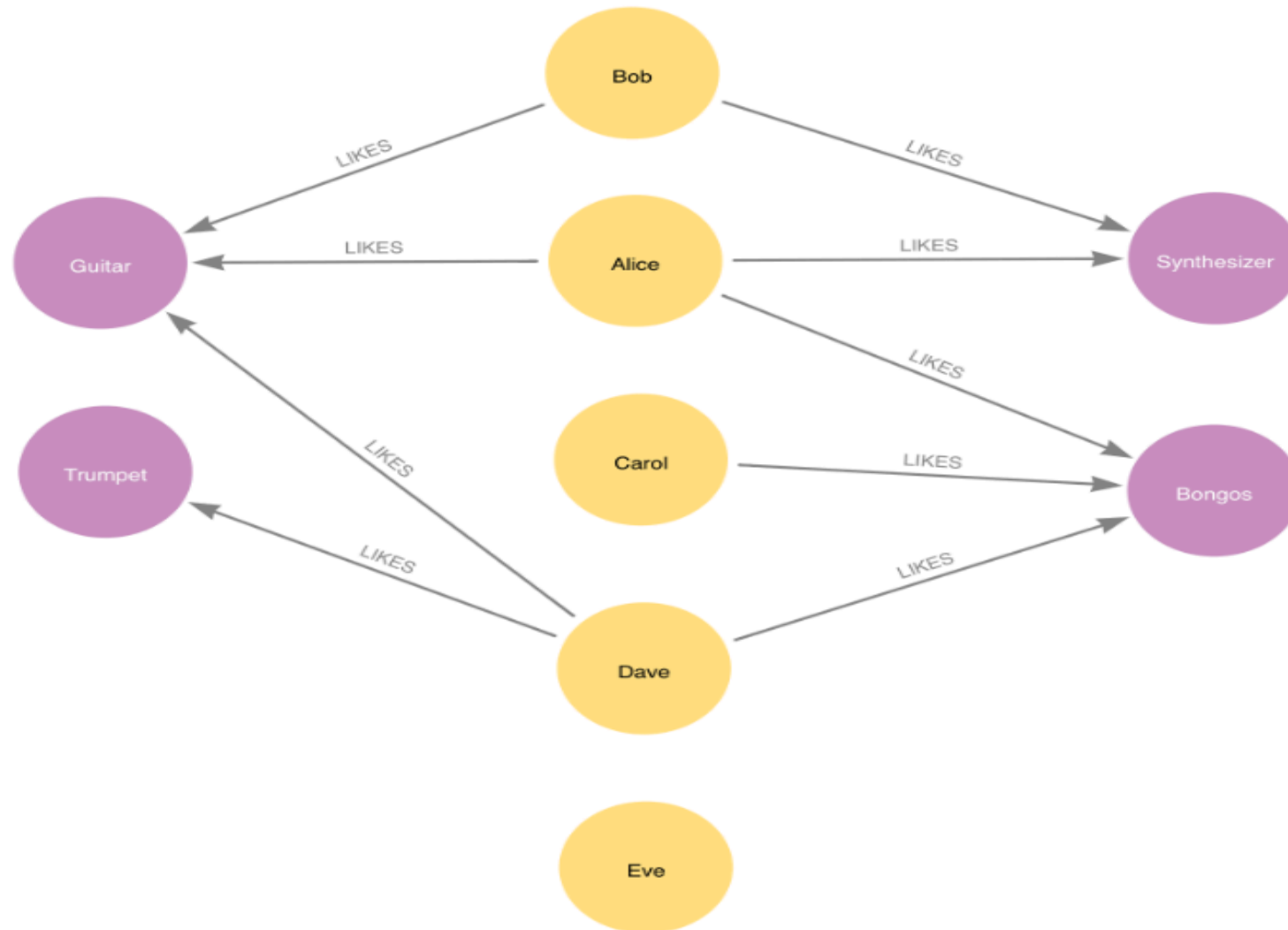
Write Mode

```
CALL gds.nodeSimilarity.write('myGraph', {  
  writeRelationshipType: 'SIMILAR',  
  writeProperty: 'score'  
})  
YIELD nodesCompared, relationshipsWritten
```

Persists `(:Person)-[:SIMILAR {score: 0.66}]->(:Person)` directly to Neo4j

Relationships are always **directed**

Example Graph



Code

```
CREATE
  (alice:Person {name: 'Alice'}),
  (bob:Person {name: 'Bob'}),
  (carol:Person {name: 'Carol'}),
  (dave:Person {name: 'Dave'}),
  (eve:Person {name: 'Eve'}),
  (guitar:Instrument {name: 'Guitar'}),
  (synth:Instrument {name: 'Synthesizer'}),
  (bongos:Instrument {name: 'Bongos'}),
  (trumpet:Instrument {name: 'Trumpet'}),
  (alice)-[:LIKES]->(guitar),
  (alice)-[:LIKES]->(synth),
  (alice)-[:LIKES {strength: 0.5}]->(bongos),
  (bob)-[:LIKES]->(guitar),
  (bob)-[:LIKES]->(synth),
  (carol)-[:LIKES]->(bongos),
  (dave)-[:LIKES]->(guitar),
  (dave)-[:LIKES {strength: 1.5}]->(trumpet),
  (dave)-[:LIKES]->(bongos);
```

The following statement will project the graph and store it in the graph catalog.

```
MATCH (source:Person)
OPTIONAL MATCH (source)-[r:LIKES]->(target:Instrument)
RETURN gds.graph.project(
    'myGraph',
    source,
    target,
    { relationshipProperties: r { strength: coalesce(r.strength, 1.0) } }
)
```

The following will run the algorithm, and stream results:

```
CALL gds.nodeSimilarity.stream('myGraph')
YIELD node1, node2, similarity
RETURN gds.util.asNode(node1).name AS Person1, gds.util.asNode(node2).name AS Person2, similarity
ORDER BY similarity DESCENDING, Person1, Person2
```

Table 14. Results

Person1	Person2	similarity
"Alice"	"Bob"	0.6666666666666666
"Bob"	"Alice"	0.6666666666666666
"Alice"	"Dave"	0.5
"Dave"	"Alice"	0.5
"Alice"	"Carol"	0.3333333333333333
"Carol"	"Alice"	0.3333333333333333
"Carol"	"Dave"	0.3333333333333333
"Dave"	"Carol"	0.3333333333333333
"Bob"	"Dave"	0.25
"Dave"	"Bob"	0.25

Limiting Results

Strategy	Parameter	Effect
Top K per node	topK: 1	Best match per node
Bottom K per node	bottomK: 1	Worst match per node
Global top N	topN: 3 + topK: 1	Top 3 overall
No limit	topK: 10 , topN: 0	All pairs (default)

The following will run the algorithm, and stream the top 1 result per node:

```
CALL gds.nodeSimilarity.stream('myGraph', { topK: 1 })
YIELD node1, node2, similarity
RETURN gds.util.asNode(node1).name AS Person1, gds.util.asNode(node2).name AS Person2, similarity
ORDER BY Person1
```

Table 19. Results

Person1	Person2	similarity
"Alice"	"Bob"	0.666666666666667
"Bob"	"Alice"	0.666666666666667
"Carol"	"Alice"	0.333333333333333
"Dave"	"Alice"	0.5

The following will run the algorithm, and stream the 3 highest out of the top 1 results per node:

```
CALL gds.nodeSimilarity.stream('myGraph', { topK: 1, topN: 3 })
YIELD node1, node2, similarity
RETURN gds.util.asNode(node1).name AS Person1, gds.util.asNode(node2).name AS Person2, similarity
ORDER BY similarity DESC, Person1, Person2
```

Table 21. Results

Person1	Person2	similarity
"Alice"	"Bob"	0.6666666667
"Bob"	"Alice"	0.6666666667
"Dave"	"Alice"	0.5

Weighted Similarity

Weighted Jaccard:

$$J_w(A, B) = \frac{\sum_i \min(a_i, b_i)}{\sum_i \max(a_i, b_i)}$$

Weighted Overlap:

$$O_w(A, B) = \frac{\sum_i \min(a_i, b_i)}{\min(\sum_i a_i, \sum_i b_i)}$$

Cosine naturally handles weighted input

The following query will respect relationship properties in the similarity computation:

```
CALL gds.nodeSimilarity.stream('myGraph', { relationshipWeightProperty: 'strength', similarityCutoff: 0.3 })
YIELD node1, node2, similarity
RETURN gds.util.asNode(node1).name AS Person1, gds.util.asNode(node2).name AS Person2, similarity
ORDER BY Person1
```

Table 24. Results

Person1	Person2	similarity
"Alice"	"Bob"	0.8
"Alice"	"Dave"	0.3333333333333333
"Bob"	"Alice"	0.8
"Dave"	"Alice"	0.3333333333333333

It can be seen that the similarity between Alice and Dave decreased (from 0.5 to 0.33) compared to the non-weighted version of this algorithm.

Alice likes Guitar, Synthesize and Bongos with strengths (1, 1, 0.5). Dave likes Guitar, Bongos and Trumpet with strengths (1, 1, 1.5). Therefore, taking Alice and Dave's neighbours, we have list of strengths for Alice as $A = (1, 1, 0.5, 0)$ and for Dave $B = (1, 0, 1, 1.5)$, indexed as Guitar, Synthesizer, Bongos, Trumpet.

The weighted (Jaccard) node similarity of Alice and Dave is hence:

$$J_W(A, B) = \frac{1 + 0 + 0.5 + 0}{1 + 1 + 1 + 1.5} = \frac{1}{3}$$

Performance Optimization

`useComponents` parameter:

- `false` (default) — computes across entire graph
- `true` — computes components first, skips cross-component comparisons
- `<String>` — uses pre-computed component node property

`degreeCutoff` + `upperDegreeCutoff` : filter nodes by degree range

`similarityCutoff` : discard low-score pairs early

Key Takeaways

- Node Similarity: **neighbor-based comparison** on bipartite graphs
- Three metrics: **Jaccard**, **Overlap**, **Cosine**
- Memory bound by $O(n^2)$ — use `topK` / `cutoff` to manage
- Weighted version uses relationship properties
- Four execution modes: `stream`, `stats`, `mutate`, `write`
- Always produces **directed** similarity relationships

References

- [Node Similarity Documentation](#)
- [Jaccard Index — Wikipedia](#)
- [Overlap Coefficient — Wikipedia](#)
- [Structural Equivalence — Wikipedia](#)
- [Bipartite Graphs — Wikipedia](#)

